

Smart Travel

Delaram Doroudgarian - 5881909

delaramdoroud82@gmail.com

Mobile Development course

Table of Contents

Project Overview	2
Functional Requirements	2
Use case diagram	3
Screen mockups	3
Technical Architecture and Implementation Plan	13
Backend and Data Management	14
Data Persistence	14
State Management.....	15
Hardware Features Used	15
External Libraries/Dependencies	16
Development Challenges and Solutions	17
UI/UX Challenges.....	17
Smart Travel Recommendation Logic	17
Round-Trip Transportation Flow	17
Maps and Location Handling.....	17
Booking Availability and Capacity Management	17
Use of Generative AI (LLMs) in This Project	18
Testing and Evaluation.....	18
Conclusion and Future Improvements	19
Summary of Achievements	19
Potential Future Improvements.....	19
References:.....	20

Project Overview

Smart Travel is a Flutter-based tourism application designed to help users explore destinations, compare transportation options, find suitable accommodations, view available tours and attractions, and manage their reservations in one place. The application uses Firebase services to store and manage travel-related data, user reservations, reviews, image paths, and other dynamic content.

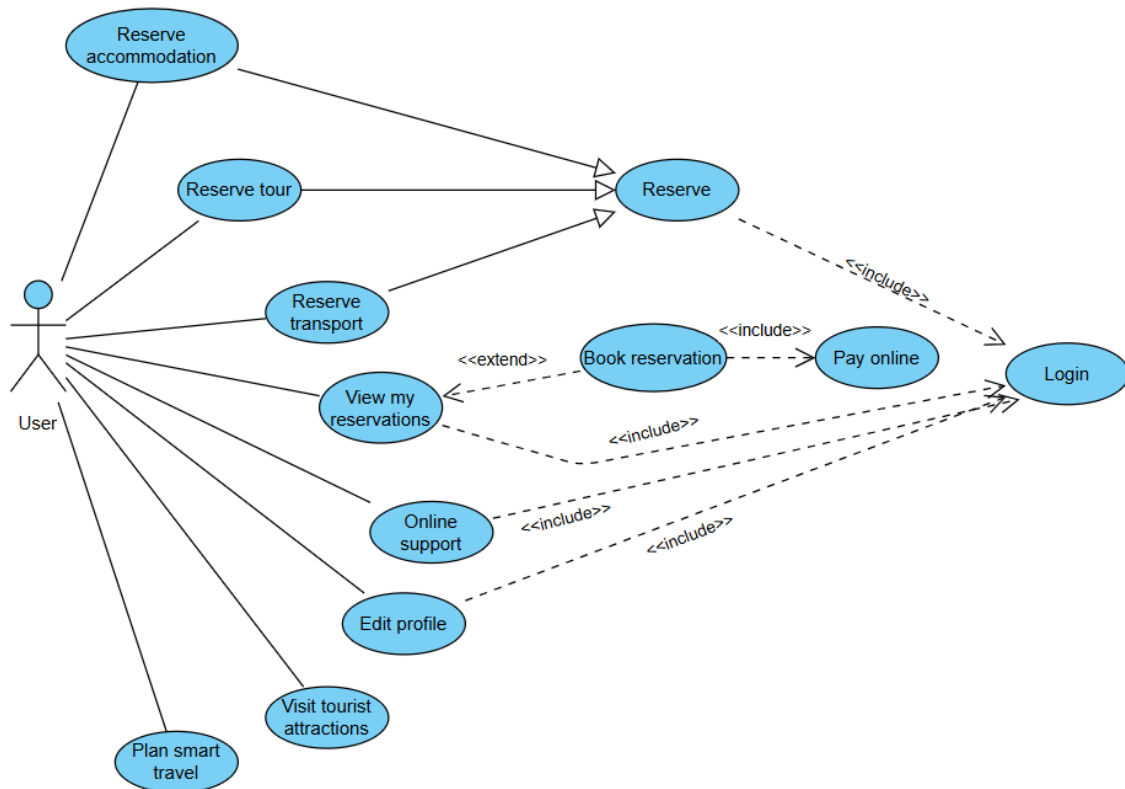
The main goal of this project is to provide a smoother and more centralized travel planning experience. Instead of using multiple separate platforms for transportation, accommodation, tours, and destination discovery, users can access the main travel-related services from a single mobile application. From the home page, users can browse recommended transports, accommodations, and tours, open detailed pages for each item, and reserve suitable options directly through the app.

One of the key implemented features is the Smart Travel Planner. This feature allows users to select an origin, destination, budget, trip duration, and number of travelers. Based on this information, the application recommends suitable transportation and accommodation options, displays them as interactive cards, and allows users to open the related detail pages to continue the booking or reservation process.

Functional Requirements

The Smart Travel application includes several functional requirements that support the main travel planning and reservation flow. These requirements define the core features that users can access in the application and describe how the system responds to user actions.

Use case diagram

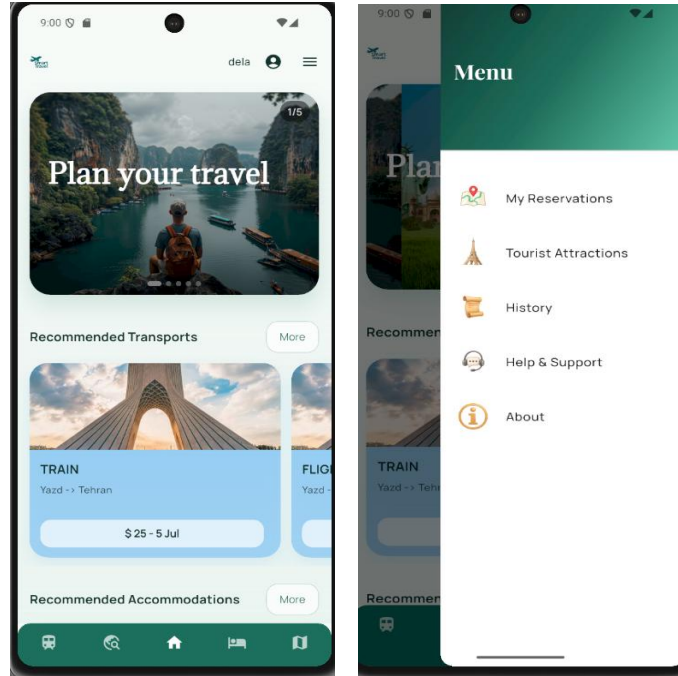


Screen mockups

- Home Page

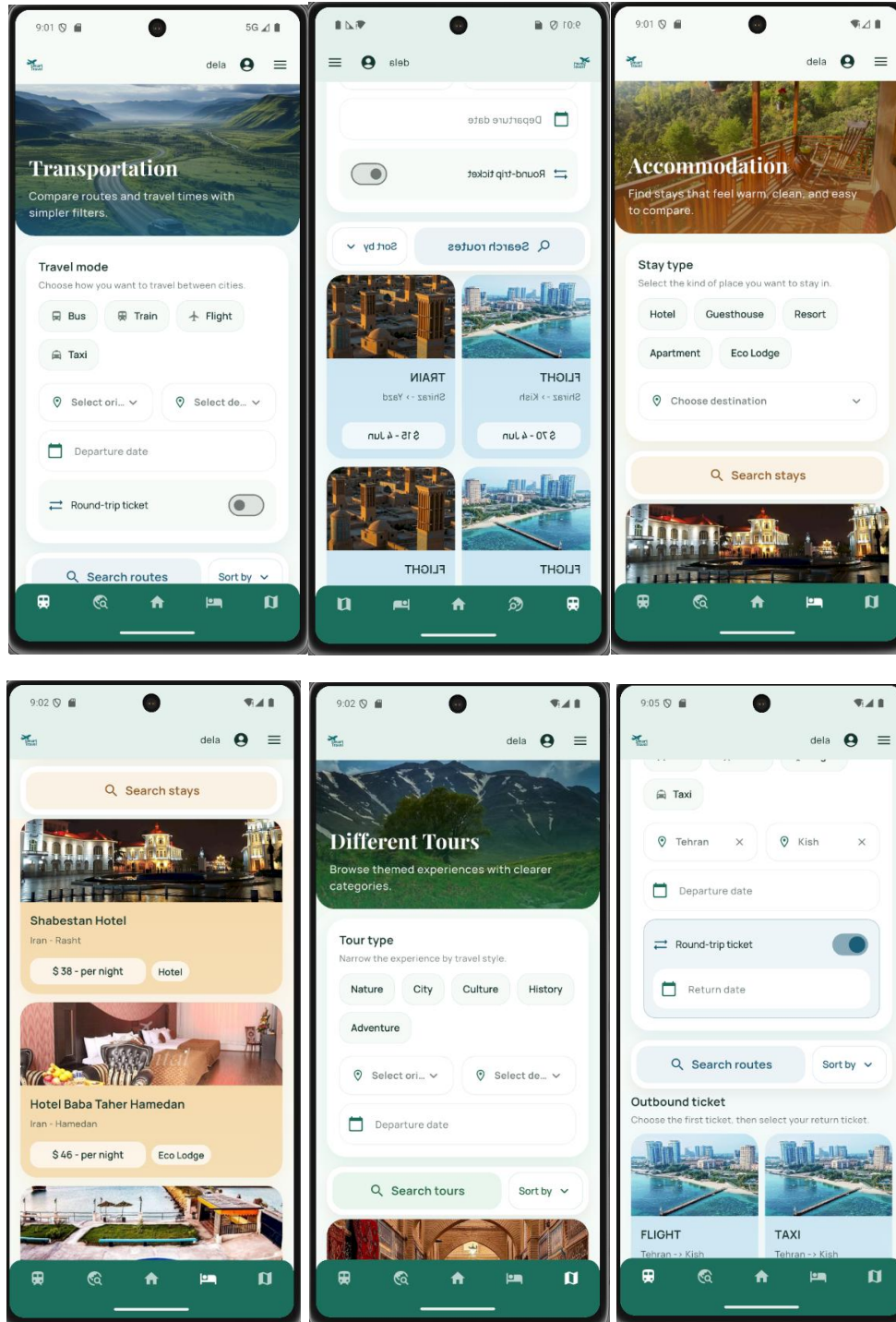
The Home page provides users with quick access to the main travel services in the application. It displays recommended transports, accommodations, and tours using horizontal card sections. Each card shows summary information such as image, title, destination or route, price, and date when available. Users can tap a card to open its detail page or use the More button to view the full list of items for that category.

The page also includes a navigation bar with a hamburger menu on the right side. Opening this menu displays additional options, including My Reservations, History, Tourist Attractions, Help & Support, and About. These options allow users to access saved reservations, browse attractions, request support, and view general information about the application.



- Reservation Page

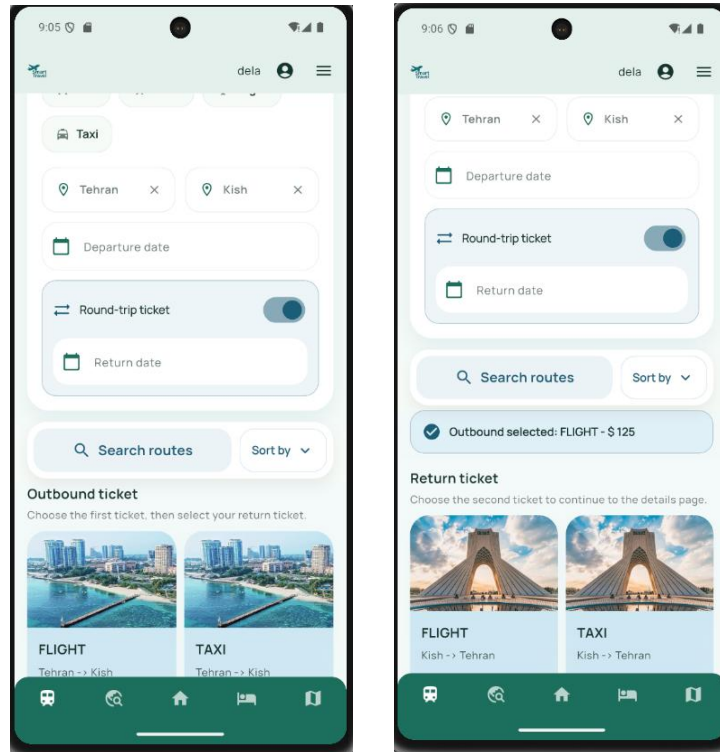
The Reservation page allows users to browse and compare available travel services. Depending on the selected category, users can view transportation options, accommodations, or tours. The page supports filtering, while transports and tours also support sorting by price or date. Accommodations are displayed by price by default. Selecting a card opens the related detail page for more information and booking.



- Transportation Reservation / Round Trip Flow

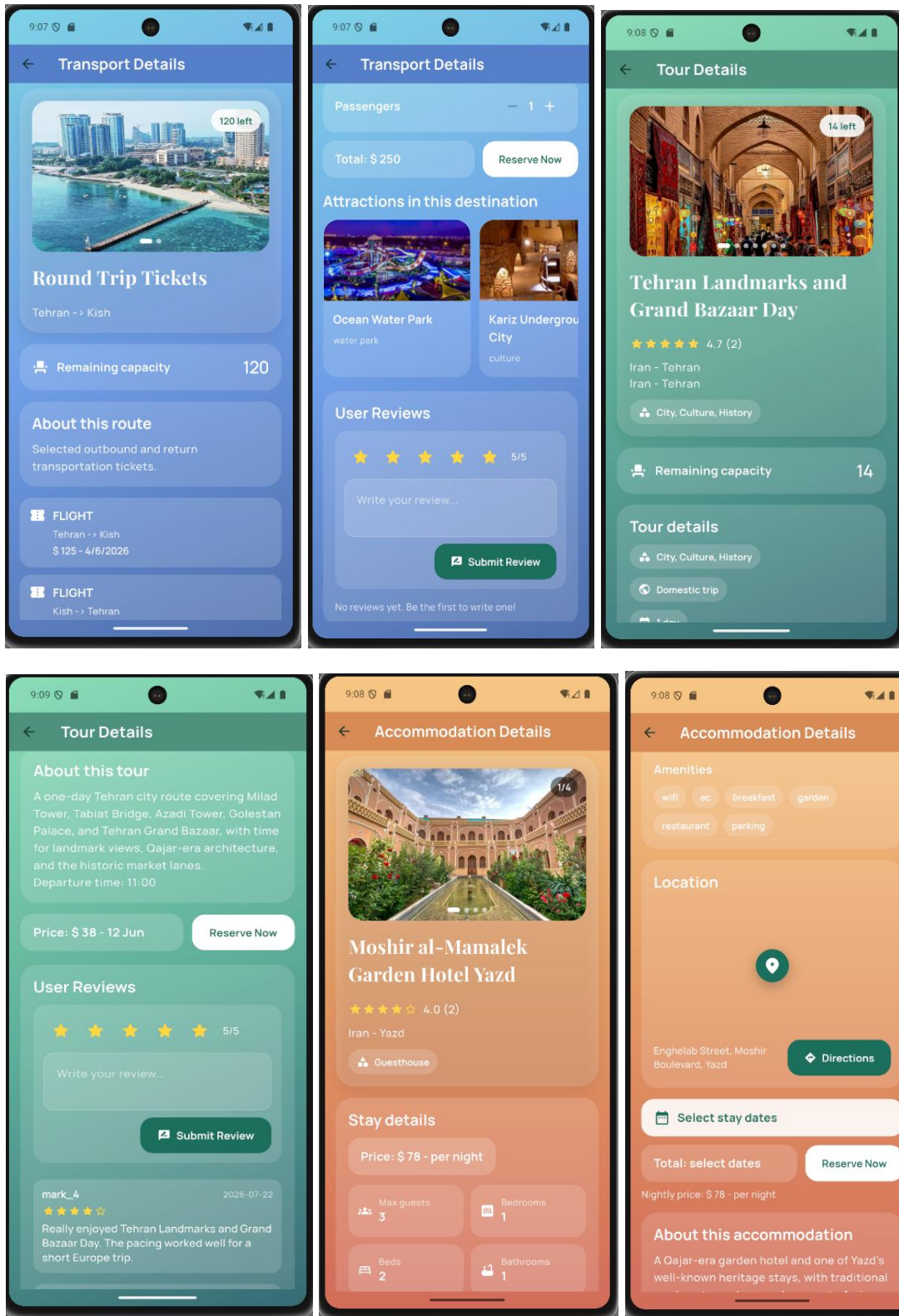
The transportation reservation flow allows users to search for available routes between cities. Users can choose a one-way trip or enable round-trip mode. In round-trip mode, the user first selects an outbound ticket and then selects a return ticket. After both tickets are selected, the

application opens the detail page with the combined round-trip information ready for booking.



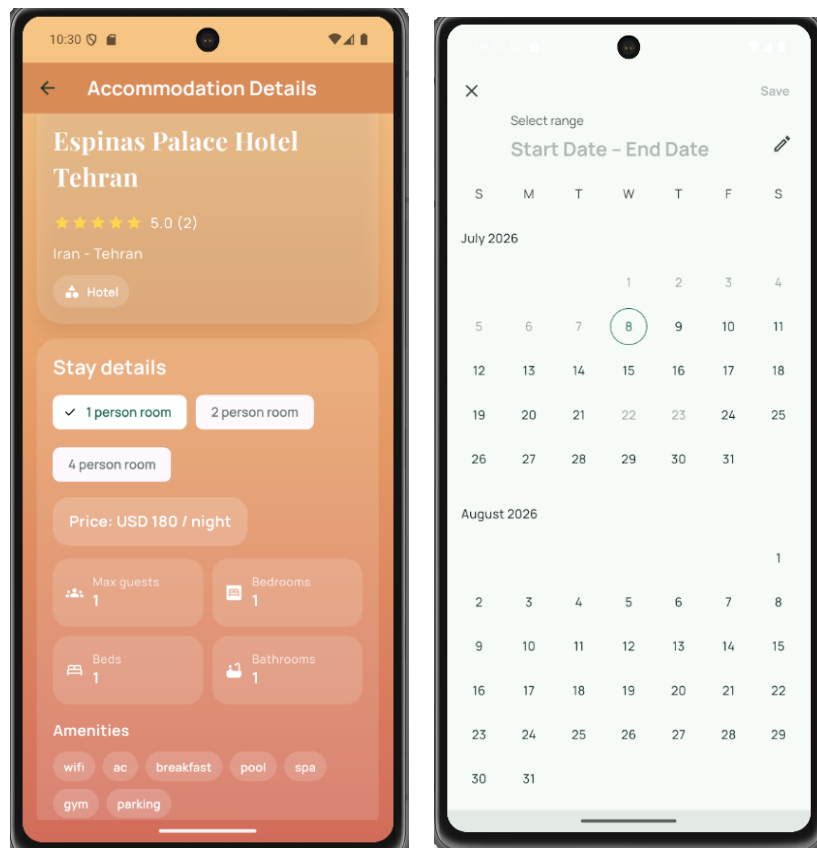
- Details Page

The Details page presents complete information about the selected travel item. It displays images, title, price, route or destination information, description, ratings, related content, and a reviews section where users can view existing feedback or submit their own rating and comment. For accommodations, it also shows stay details such as maximum guests, bedrooms, beds, bathrooms, amenities, room options, and location. For transports, users can select the number of passengers, and the total price updates based on the passenger count. The page also displays the remaining capacity for transports and tours. Users can complete the reservation process from this page.



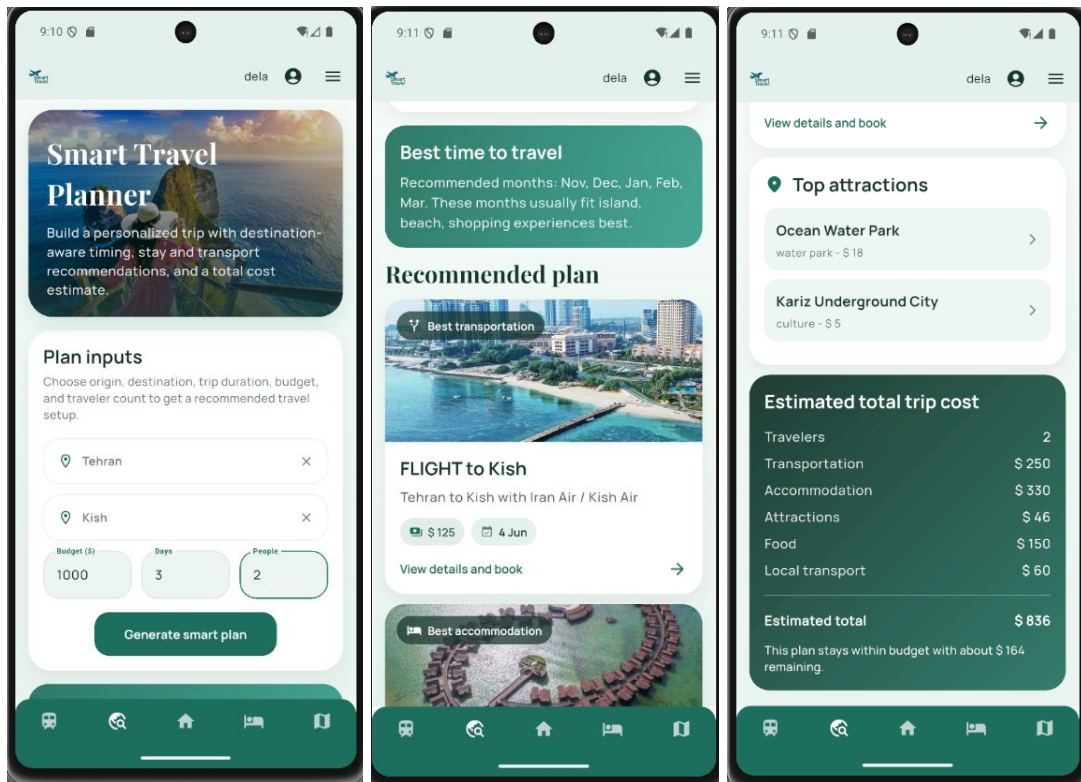
- Accommodation Details / Booking

The accommodation details section shows the main stay information, including nightly price, guest capacity, bedrooms, beds, bathrooms, and amenities. It also includes a location section with a map preview and directions support, helping users understand where the accommodation is located before booking. Users can select their stay dates before booking. If the accommodation has different room options, the user can choose a room type and the total price is calculated based on the selected dates and room price. Booked accommodation dates are checked against existing finalized bookings. If the selected room type or stay is no longer available for the selected dates, the application prevents the booking.



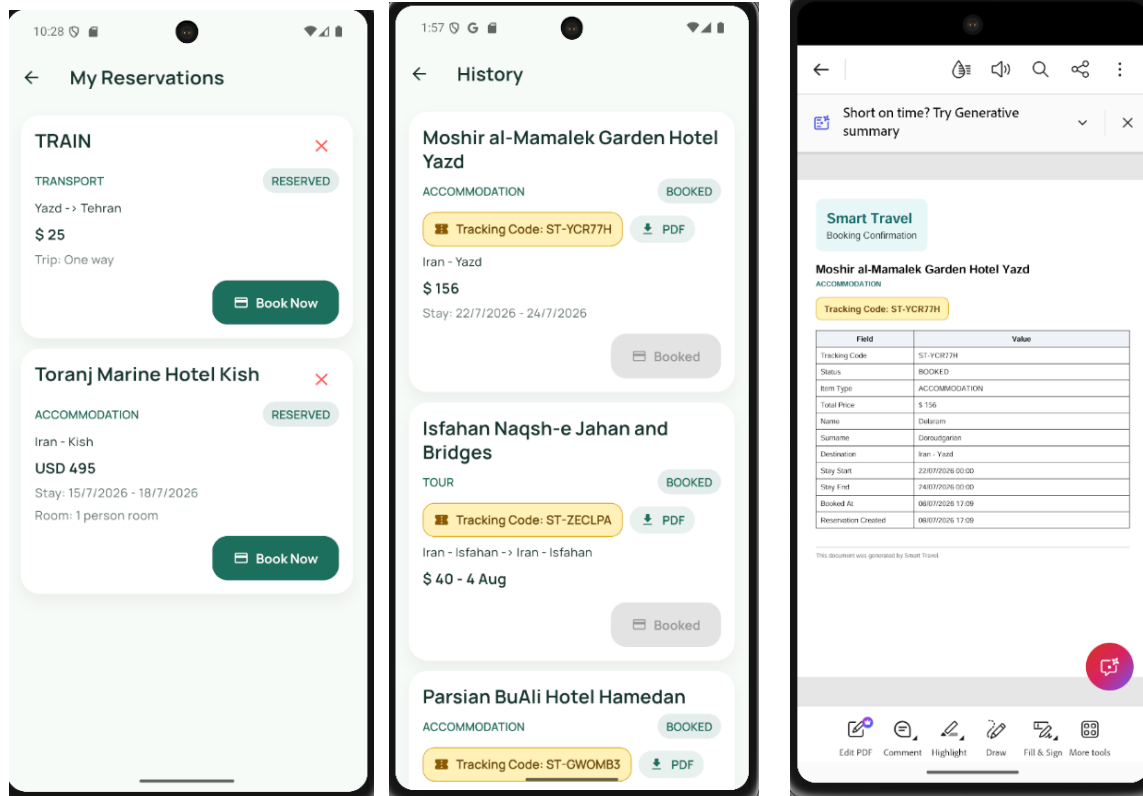
- Smart Travel Planner

The Smart Travel Planner helps users generate a personalized travel plan. Users enter their origin, destination, budget, trip duration, and number of travelers. Based on these inputs, the application recommends suitable transportation and accommodation options, estimates the total trip cost, and displays the recommended items as interactive cards that can be opened for details and booking.



- My Reservations and History Page

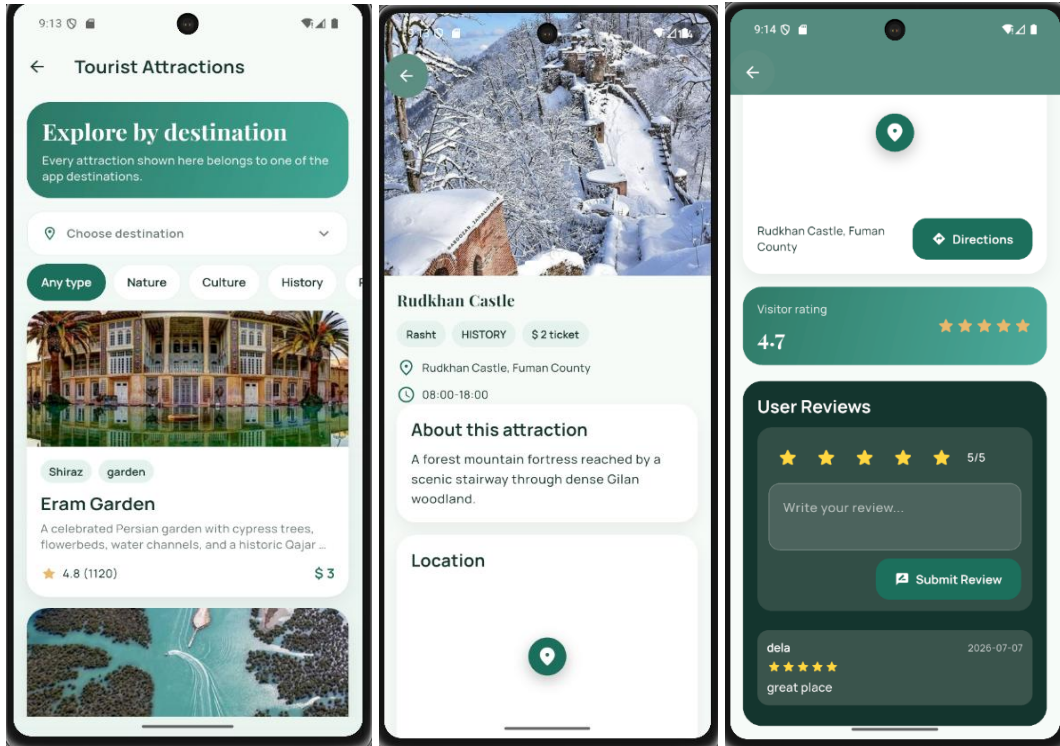
The My Reservations page shows the user's saved reservations and allows them to review or remove active reserved items. The History page shows completed bookings after the user clicks Book Now. Booked items include a tracking code and a PDF download option for saving or sharing the booking confirmation.



- Attractions and Attraction Details Page

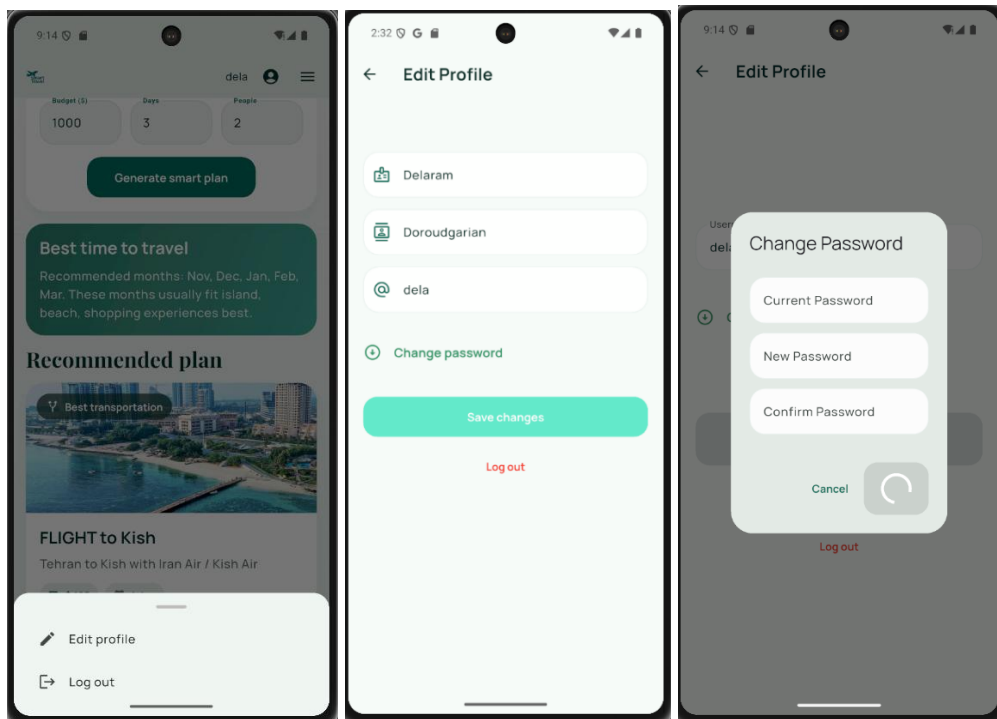
The Attractions page allows users to explore available attractions for different destinations. Each attraction includes visual content and basic information such as name, category, and ticket price. Users can open an attraction to view more detailed information.

The Attraction Details page provides detailed information about a selected attraction, including images, description, category, ticket information, and location details. The page also includes a map preview and directions support, allowing users to see where the attraction is located and open navigation if needed.



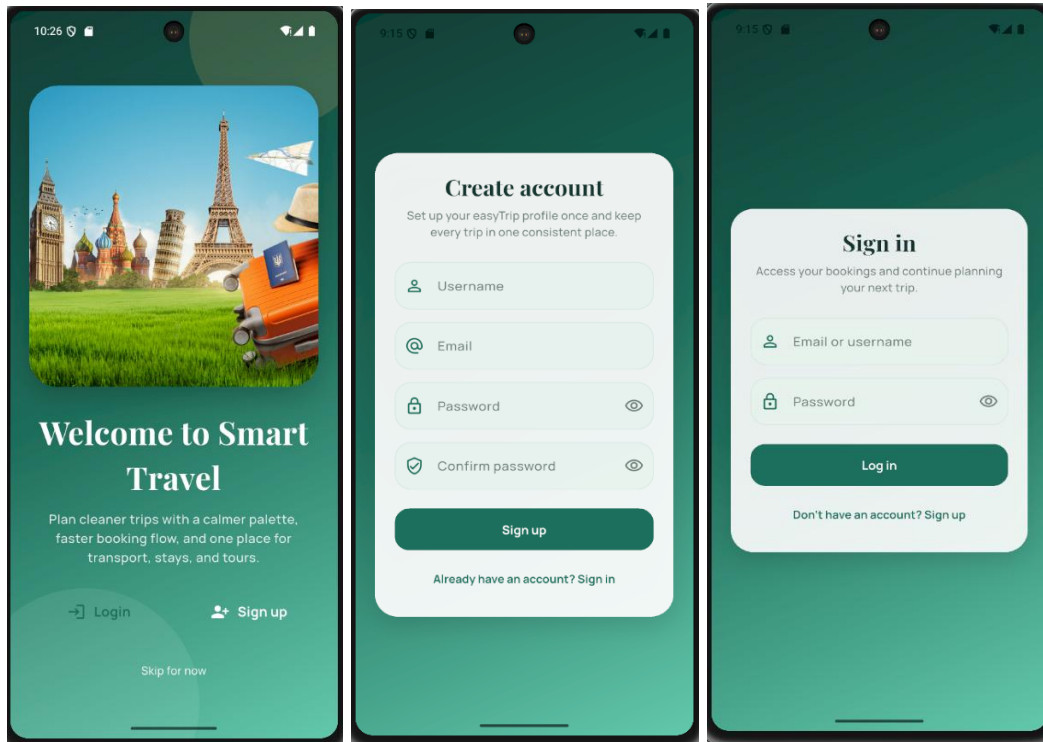
- User Account / Profile Page

The User Account page allows users to view and manage their profile information. It provides access to account-related actions such as editing profile details, viewing reservations, and managing the user experience within the application.



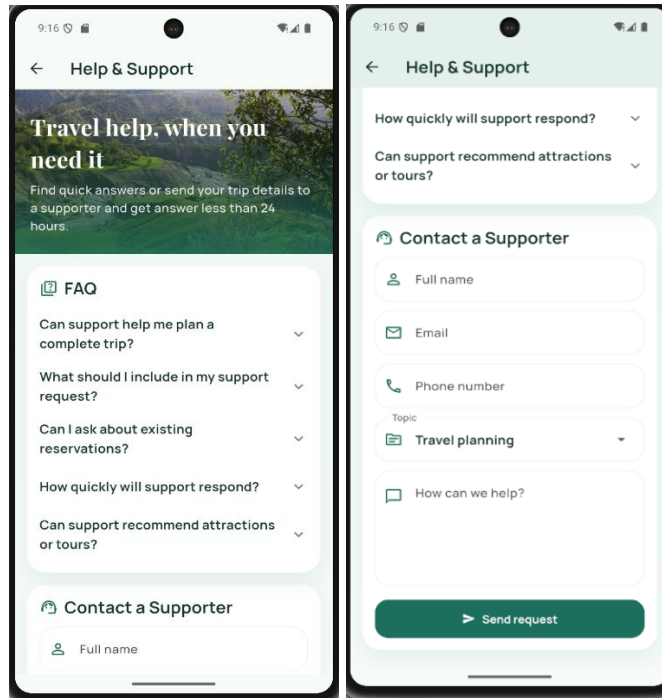
- Login and Register Pages

The Login and Register pages handle user authentication. New users can create an account, while existing users can sign in to access reservation features, user-specific data, and saved bookings. Authentication ensures that reservations and reviews are connected to the correct user.



- Online Support Page

The Online Support page allows users to request help related to travel planning, reservations, or general application usage. Users can describe their issue or question so that support can assist them with their travel-related needs.



Technical Architecture and Implementation Plan

- **Platform**

Smart Travel was developed using the Flutter framework and the Dart programming language. Flutter was selected because it allows the application to run on multiple supported platforms from a single codebase while maintaining a consistent user interface and development structure. This is especially useful for a tourism application, where the same design, navigation flow, and reservation experience should remain consistent across devices.

The application uses Firebase as its backend service. Firebase provides authentication, Firestore database storage, and real-time data updates. This allows the application to manage users, travel items, reservations, reviews, and application content without requiring a separate custom backend server.

The project follows a structured Flutter architecture by separating screens, reusable components, widgets, models, data access logic, and theme-related files. This separation makes the application easier to maintain and extend. For example, UI elements such as travel cards, carousel sections, form fields, and detail layouts are reused across different parts of the application, while model classes are used to represent travel-related data in a cleaner and more consistent way.

- **Reason for Technology Choices**

Cross-Platform Development: Flutter allows the application to be developed for multiple platforms using a single codebase. This reduces development time and helps keep the user experience consistent across devices.

Reusable UI Components: Flutter's widget-based structure makes it suitable for building reusable interface components such as travel cards, carousel sections, detail pages, input fields, and booking controls. Components like `DetailsFrame`, `HomeSectionCarousel`, and shared input widgets help reduce repeated code and keep the UI consistent.

Fast Development Process: Features such as Hot Reload and Hot Restart make it easier to test UI changes quickly during development. This was useful when improving the home page cards, reservation screens, Smart Travel Planner, and detail page layouts.

Firestore Integration: Firestore was used to manage backend functionality, including user authentication, Firestore collections, reservations, reviews, and image-related data. This simplified backend development and allowed the application to work with dynamic travel content.

Dynamic Data Handling: Firestore allows the application to retrieve and display updated data for transports, accommodations, tours, attractions, reservations, and reviews. This makes the application flexible because travel content can be updated from the backend without changing the app code.

Model-Based Data Structure: Firestore data is converted into Dart model objects such as `Transport`, `Accommodation`, `Tour`, `Attraction`, and `Destination`. This makes the code cleaner, improves readability, and reduces direct map-reading inside UI files.

Scalable Feature Structure: The project separates screens, models, components, widgets, data access, and theme files. This structure makes it easier to add new features such as Smart Travel recommendations, support pages, review sections, and improved reservation flows.

Backend and Data Management

Data Persistence

The Smart Travel application uses cloud-based storage solutions to store and manage dynamic travel data. Firestore is used as the main backend platform for handling authentication, database records, reservations, reviews, booking status, tracking codes, and image path metadata.

- **Cloud Database:** Cloud Firestore is used as the primary database for storing destinations, transports, accommodations, tours, attractions, user reservations, reviews,

capacity values, and booking-related records. The application uses Firestore collections to organize travel-related data and retrieve it through queries and streams.

- **User Reservations:** User bookings are stored in the `my_reservations` collection. Each reservation includes information such as the user ID, selected item, item type, price, dates, transport tickets, room details, booking status, tracking code, and other booking-related data.
- **Reviews and Ratings:** Reviews are stored separately and connected to items using `targetType` and `targetId`. This allows the application to manage ratings and comments for different item types such as accommodations, transports, tours, and attractions.
- **Image Data:** Travel item images are stored locally in the `images/` folder, while their image paths are saved in Firestore and used by the app to display the correct visuals.

State Management

The project mainly uses Flutter's built-in state management approach with `StatefulWidget`, `setState`, controllers, and streams. This approach is used to manage UI changes such as selected filters, selected destinations, selected transport tickets, form inputs, selected room options, and Smart Travel planner results.

- **Local UI State:** `setState` is used in screens such as Reservation, Smart Travel, Details Page, and form-based pages to update the interface when the user changes filters, selects tickets, chooses stay dates, or submits data.
- **Text Controllers:** `TextEditingController` is used for handling user inputs such as budget, trip duration, traveler count, login fields, registration fields, and profile editing fields.
- **Stream-Based Updates:** Firestore streams combined with `StreamBuilder` are used to display updated backend data in pages such as Home, Reservation, Smart Travel, My Reservations, Attractions, Details Page, and Reviews.

Hardware Features Used

The application uses several device-related features to support travel browsing, location awareness, and reservation-related interactions.

- **Internet Connectivity:** The app relies on internet access to communicate with Firebase services, load Firestore data, authenticate users, submit reviews, and save reservations.

- **Location Services / GPS:** The application uses the device's current location when users request directions to an accommodation or attraction. When the Directions button is pressed, the app checks whether location services are enabled, requests location permission if needed, and uses the current GPS position as the starting point for navigation.
- **External App Launching:** The app uses external application launching to open Google Maps or a compatible navigation app for route guidance.

External Libraries/Dependencies

Several external packages are used to support backend services, UI styling, formatting, and navigation features:

- **Firestore:** `firebase_core`, `firebase_auth`, `cloud_firestore`, and `firebase_app_check` are used for Firebase initialization, authentication, Firestore database access, and app verification. The `firebase_storage` package is included in the project, although the current image implementation mainly uses local image assets with paths stored in Firestore.
- **Date and Time Formatting:** `intl` is used to format dates for transports, tours, reservations, and booking-related information.
- **UI Styling:** `google_fonts` is used to apply custom typography across the application and improve the visual design.
- **Navigation and External Links:** `url_launcher` is used to open external links such as map directions or location-related navigation.
- **Location Services:** `geolocator` is used to check location service status, request location permissions, and retrieve the user's current GPS position for directions.
- **Assets and Icons:** `cupertino_icons` provides additional icon support, while the local `images/asset` folder stores visual content used for destinations, accommodations, transports, tours, and attractions.
- **PDF Generation and Sharing:** `pdf` and `printing` are used to generate booking confirmation PDFs and allow users to save or share completed booking details from the History page.

Development Challenges and Solutions

UI/UX Challenges

One of the main challenges was designing a consistent interface for different travel services such as transports, accommodations, tours, and attractions. Each item type required different information, but the application still needed a unified visual style. This was handled by creating reusable components such as DetailsFrame, HomeSectionCarousel, input fields, and shared detail page layouts. These components helped reduce repeated UI code and kept the design consistent across Home, Reservation, and Details pages.

Smart Travel Recommendation Logic

The Smart Travel Planner required extra logic because recommendations depend on multiple user inputs, including origin, destination, budget, duration, and number of travelers. A key issue was calculating accommodation cost correctly. Since accommodation prices are based on rooms or units rather than individual travelers, the calculation was updated to consider the accommodation capacity and number of required units instead of simply multiplying by the number of travelers.

Round-Trip Transportation Flow

Round-trip transportation required a different flow from one-way transport booking. The user first needs to select an outbound ticket and then a return ticket before opening the details page. This was solved by creating a separate round-trip selection flow that stores the selected outbound transport, displays it to the user, and then allows the return ticket to be selected before generating the combined transport booking details.

Maps and Location Handling

Adding directions support introduced challenges related to location permissions and device settings. The application needed to handle cases where location services were disabled, permission was denied, or permission was permanently denied. This was solved by using the geolocator package to check location service status, request permission, retrieve the current GPS position, and show appropriate messages when location access was not available.

Booking Availability and Capacity Management

Another challenge was preventing finalized bookings from being duplicated. Accommodation bookings needed date and room availability checks, while transports and tours required remaining capacity tracking. This was handled by storing bookingStatus values, checking finalized bookings before confirmation, and reducing transport or tour capacity only when the user completes the Book Now action.

Use of Generative AI (LLMs) in This Project

I first worked with Flutter several years ago, but I had not used it regularly since then. Because of this gap, generative AI was very helpful in refreshing my memory about Flutter syntax, widget structure, navigation, state handling, and common Flutter development patterns. Instead of spending a long time searching for syntax details or trying to remember how specific Flutter features worked, I was able to use AI support to move through the implementation process faster.

AI was also useful during debugging. When build errors, emulator issues, dependency problems, or implementation bugs appeared, AI helped explain the possible causes and suggested practical steps to investigate and fix them. This made the debugging process faster and more structured, especially when working with Firebase, permissions, GPS functionality, and Flutter build commands.

ChatGPT: was used during the coding and implementation phase of the project. It was also used as a support tool for UI design decisions, including improving card layouts, organizing page sections, refining form behavior, and making the Smart Travel recommendation results more user-friendly and visually consistent with the rest of the application.

Gemini: was used mainly for editing and polishing the final report. It helped refine the wording, improve clarity, and make the final document more consistent and professional before submission.

Testing and Evaluation

Manual Testing

Manual testing was carried out throughout the development process to check the main travel planning, reservation, booking, history, profile, and PDF confirmation flows. The testing focused on making sure that users could move smoothly from browsing items to reserving, booking, and viewing finalized bookings in History.

The following issues were found during manual testing and fixed:

- Reserving the same accommodation for different dates overwrote the previous reservation instead of creating a separate record.
- Booked items in the History page were still behaving like active clickable reservation items.

- The PDF confirmation did not display the user's name and surname because they were stored in the users collection, not directly inside the reservation document.
- Accommodation dates that were already booked were not always disabled correctly in the date picker.
- Book Now did not always show the expected success message after moving an item to History.
- Transport and tour bookings needed capacity checks so users could not book more seats than the remaining capacity.
- Some error messages did not use a clear red background, making them less noticeable.
- Adding new profile fields caused form validation issues because multiple forms were using the same GlobalKey.

Conclusion and Future Improvements

Summary of Achievements

The Smart Travel project successfully developed a Flutter-based tourism application that allows users to explore destinations, compare transports, view accommodations, browse tours and attractions, submit reviews, and manage reservations in one place.

The application also includes a Smart Travel Planner that recommends suitable transportation and accommodation options based on user inputs such as origin, destination, budget, trip duration, and number of travelers. Map previews and GPS-based directions were also implemented for accommodations and attractions.

Overall, the project demonstrates a functional travel planning and reservation system using Flutter, Firebase, reusable UI components, model-based data handling, and location-related features.

Potential Future Improvements

Although the current version of Smart Travel is functional, several improvements could make the application more advanced and easier to manage.

One important improvement would be enhancing the recommendation system. The current Smart Travel Planner recommends transportation and accommodation based on selected user inputs, but future versions could make this feature more intelligent by integrating an AI API. This would allow the application to generate more personalized travel suggestions based on user preferences, travel style, budget, season, ratings, and destination characteristics.

Another future improvement would be adding a complete payment system. Once the payment is successfully completed, the system could automatically send a confirmation email containing the reservation details, payment status, and booking information.

A further improvement would be developing a full admin dashboard. Currently, travel data such as destinations, accommodations, transports, tours, and attractions is managed through seed data or direct database updates. An admin dashboard would allow administrators to add, edit, and remove travel content through a user interface, reducing the need for manual database entry and making content management easier and safer.

References:

<https://flutter.dev/>

<https://firebase.google.com/docs>